Hashmap / HashSet Playlist

(01) Find Players with zero or One Losses –

Input:- matches – {{1,3}, {2,3}, {3,6}, {5,6} {5,7} {4,5}, {4,8}, {4,9}, {10,4} {10,9}}

Output → {{1,2,10}, {4,5,7,8}}

{1,2,10} — Never lost a match

{4,5,7,8} — lost only one match

— DRY RUN –

- never_lost – जो map में नहीं है {1,2,10}

lost_once → जिनकी val 1 है

lost_map

key	val
3	2
6	2
7	1 ✓
5	1 ✓
8	1 ✓
9	2
4	1 ✓

{7,5,8,4}

```
public List<List<Integer>> findWinners(int[][] matches) {
    Map<Integer, Integer> lost = new HashMap<>();

    for (int[] it : matches) {
        int lose = it[1];
        lost.put(lose, lost.getOrDefault(lose, 0) + 1);
    }
    List<Integer> notLost = new ArrayList<>();
    List<Integer> oneLos = new ArrayList<>();

    for (int[] it : matches) {
        int lose = it[1];
        int win = it[0];

        if (lost.get(lose) == 1) {
            oneLos.add(lose);
        }

        if (!lost.containsKey(win)) {
            notLost.add(win);
            lost.put(win, 2);
        }
    }

    Collections.sort(notLost);
    Collections.sort(oneLos);

    return Arrays.asList(notLost, oneLos);
}
```

③

(02) Unique Number of Occurences –

Input :- arr = {1, 2, 2, 1, 1, 3}

Output :- true.

1 → 3
2 → 2
3 → 1

occurences is unique true.

— DRY RUN — (Approach - 01) ...

① HashMap ⇒ Store occurences

1	3
2	2
3	1

⇒

② put occurences into Set

{3, 2, 1} ⇒ unique true ✓

— DRY RUN — (Approach - 02) ...

Using Given constraints –

$-1000 <= arr[i] <= 1000$

Ex. Input ⇒ -1000, -999, -1000, 3, 3, 3, 1000, 1000, 1000, 1000

arr:

0	1	2	3	4	...	1003						2000
2	1	.	.	-	...	3	0	0	0	0	0	4

-1000 + 1000 ⇒ 0

-999 + 1000 ⇒ 1

1000 + 3 ⇒ 1003

1000 + 1000 ⇒ 2000

Sort ⇒

0	0	0	0	1	2	3	4

2000

(A) Code :-

```
public boolean uniqueOccurrences(int arr[]){
    HashMap<Integer, Integer> map = new HashMap<>();

    for (int x : arr){
        map.put(x, map.getOrDefault(x,0) + 1);
    }
    HashSet<Integer> set = new HashSet<>();

    for (int freq : map.values()) {
        if(set.contains(freq)) {
            return false;
        }
        set.add(freq);
    }
    return true;
}
```

```
public boolean uniqueOccurrences(int arr[])
    int vec[] = new int[2001];
    for(int x : arr) {
        vec[x + 1000]++;
    }
```

```
public boolean uniqueOccurrences(int arr[]){
    int vec[] = new int[2001];

    for (int x : arr) {
        vec[x + 1000]++;
    }
    Arrays.sort(vec);

    for (int i = 1; i<2001 ; i++) {
        if(vec[i] != 0 && vec[i] == vec[i-1]){
            return false;
        }
    }
    return true;
}
```

```
for(int i = 0; i<2001; i++){
    if(vec[i] == 0) {
        continue;
    }
    int idx = Math.abs(vec[i]);
```

```
if(vec[idx] < 0) {
    return false;
}
vec[idx] = -1;
}
return true;
```

(5) (03) Word Pattern - (Microsoft, Amazon, makemytrip) ...

Input :- Pattern - "abba", S= "dog cat cat dog"

Output - true.

— DRY RUN —

(1) S => | dog | cat | cat | dog | countwords = 4

if (pattern.length != countWords) {
    return false;
}

map (String, char)

dog	a
cat	b

True...

(2) pattern - "abba", S= "dog cat cat fish"

S => | dog | cat | cat | fish | countwords => 4

map

dog	a
cat	b

set <character> used

a, b

return false.

(6) Code:-

```
public boolean wordPattern(String pattern, String s) {
    String words[] = s.split(" ");
    int n = pattern.length();

    if(words.length != n) {
        return false;
    }

    Map<String, Character> map = new HashMap<>();
    Set<Character> used = new HashSet<>();

    for(int i=0; i<n; i++){
        String word = words[i];
        char ch = pattern.charAt(i);

        if(!map.containsKey(word) && !used.contains(ch)){
            map.put(word, ch);
            used.add(ch);
        } else if(map.containsKey(word) &&
                  map.get(word) != ch) {
            return false;
        } else if(!map.containsKey(word) &&
                  used.contains(ch)) {
            return false;
        }
    }
    return true;
}
```

(7) ~~(01)~~ Approach-02

Pattern = "abba", s = "dog cat cat fish"

charToIndex

a	1
b	3

wordToIndex

dog	1
cat	3

Storing index+1 because to avoid 0 index case ⇒ like a → present in oth

When fish comes ⇒ a → 0

fish ⇒ word में नहीं है 0 will return true which is incorre

Code:—

```
public boolean wordPattern(String pattern, String s){
    Map<Character, Integer> charToIndex = new HashMap<>();
    Map<String, Integer> wordToIndex = new HashMap<>();

    String words[] = s.split(" ");
    int n = pattern.length();

    if(words.length != n){
        return false;
    }

    for(int i=0; i<n; i++){
        char ch = pattern.charAt(i);
        String word = words[i];

        int charIndex = charToIndex.getOrDefault(ch, 0);
        int wordIndex = wordToIndex.getOrDefault(word, 0);

        if(charIndex != wordIndex){
            return false;
        }
        charToIndex.put(ch, i+1);
        wordToIndex.put(word, i+1);
    }
    return true;
}
```

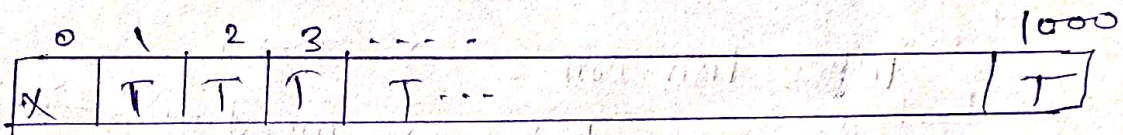
(04) Smallest Number in Infinite Set :-

Input :- ["SmallestInfiniteSet", "addBack", "popSmallest", "popSmallest", "popSmallest", "addBack", "popSmallest", "popSmallest", "popSmallest"]

Output - [null, null, 1, 2, 3, null, 1, 4, 5]

Approach - 01

1 to 1000 already stored hai ⇒ constraints of 1000 ka hai.



Initially

Code :-

```
class Smallest {
    private boolean[] nums;
    private int i;

    public Smallest() {
        nums = new boolean[1001];
        Arrays.fill(nums, true);
        i = 1;
    }

    public int popSmallest() {
        int result = i;
        nums[i] = false;
        for (int j = i+1; j<1001; j++) {
            if (nums[j]) {
                i = j;
                return result;
            }
        }
    }
```

popSmallest() - 1
popSmallest() - 2
---() = 3
addBack(2) ←

```
        i = 1001;
        return result;
    }

    public void addBack(int num) {
        nums[num] = true;
        if (num < i) {
            i = num;
        }
    }
}
```

(9) Code - (Approach - 03) -

```
class SmallestInfiniteSet {
    private TreeSet<Integer> set;
    private int currSmallest;

    public SmallestInfiniteSet() {
        set = new TreeSet<>();
        currSmallest = 1;
    }

    public int popSmallest() {
        int result;
        if (!set.isEmpty()) {
            result = set.first();
            set.remove(result);
        }
        else {
            result = currSmallest;
            currSmallest++;
        }
        return result;
    }

    public void addBack(int num) {
        if (num >= currSmallest || set.contains(num)) {
            return;
        }
        set.add(num);
    }
}
```

(10)

(05) Equal Row and Column Pairs (Microsoft & Google) — ...

Input:-

	0	1	2	3
0	3	1	2	2
1	1	4	4	5
2	2	4	2	2
3	2	4	2	2

same

Output:- 3

{3,1,2,2} (0,0)
{2,4,2,2} (2,2)
{2,4,2,2} (3,2)

— Brute force —

① // iterate → row → map O(n)
② // iterate column wise →

Code:-

```
public int equalPairs(int[][] grid) {
    int n = grid.length;
    int count = 0;

    for(int r = 0; r<n; r++){
        for(int c = 0; c<n; c++){
            boolean isEqual = true;

            for(int i = 0; i<n; i++){
                if(grid[r][i] != grid[i][c]){
                    isEqual = false;
                    break;
                }
            }
            if(isEqual){
                count++;
            }
        }
    }
    return count;
}
```

(11) Code - Optimal -

```
public int equalPairs(int[][] grid) {
    int n = grid.length;
    int count = 0;

    Map<List<Integer>, Integer> map = new HashMap<>();

    for (int row = 0; row < n; row++) {
        List<Integer> rowList = new ArrayList<>();
        for (int val : grid[row]) {
            rowList.add(val);
        }
        map.put(rowList, map.getOrDefault(rowList, 0) + 1);
    }

    for (int c = 0; c < n; c++) {
        List<Integer> colList = new ArrayList<>();
        for (int r = 0; r < n; r++) {
            colList.add(grid[r][c]);
        }
        count += map.getOrDefault(colList, 0);
    }

    return count;
}
```

(12)

(06) Count Nice Pairs in an Array –

nums = {42, 11, 1, 97}

Output = 2

Explain – {42, 97}

42 + reverse(97) = 42 + 79 = 121

{11, 1}

97 + reverse(42) = 97 + 24 = 121

– DRR RUN –

> "Always try to regroup i & j on either side then see if it makes things easy"

① for (i = 0 → n) {

nums[i] = nums[i] – rev(nums[i]);

nums[i] + rev(nums[j]) ==

nums[j] + rev(nums[i])

⇓

nums[i] – rev(nums[i]) ==

nums[j] – rev(nums[j])

② Code:–

```
class Solution {
    final int M = 1000000007;
    public int reverse(int num) {
        int rev = 0;
        while (num > 0) {
            int rem = num % 10;
            rev = rev * 10 + rem;
            num /= 10;
        }
        return rev;
    }
    public int countNicePairs(int nums[]) {
        int n = nums.length;
        Map<Integer, Integer> mp = new HashMap<>();
        for (int i = 0; i < n; i++) {
            nums[i] = nums[i] - reverse(nums[i]);
        }
        int result = 0;
        for (int i = 0; i < n; i++) {
            result = (int) ((result +
                mp.getOrDefault(nums[i], 0)) % M);
            mp.put(nums[i],
                mp.getOrDefault(nums[i], 0) + 1);
        }
        return result;
    }
}
```

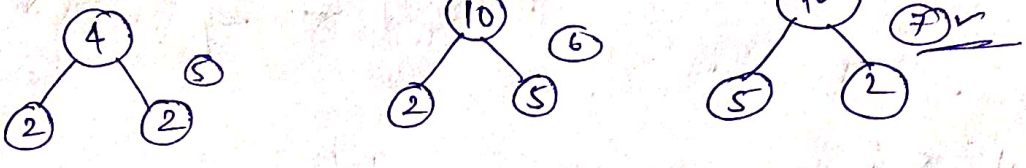
(07) Binary Tree With Factors -

Example:- arr = {2, 4, 5, 10}

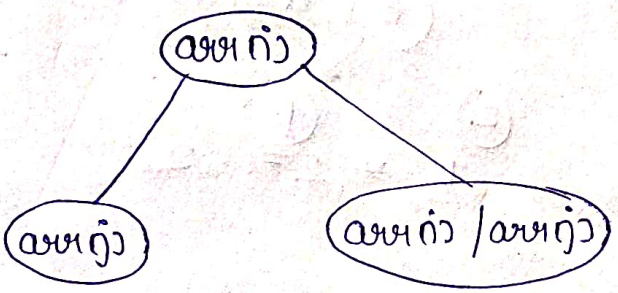
Output - 7

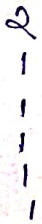
Explain:-

[2] [4] [5] [10] ⇒ (4) ✓



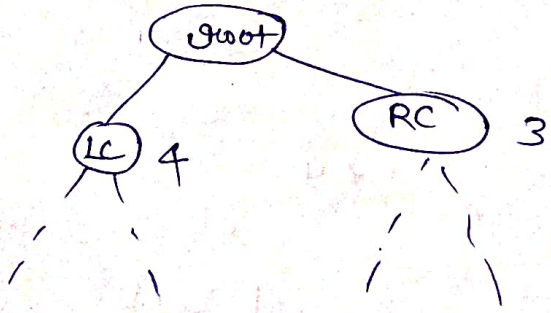
Intution ... {4, 2, 5, 10}





{4} {4,2,2} {2} {5} {10} {10,2,5} {10,5,2}

बड़े element को left child नहीं बना सकते, तो Sort कर देते हैं।



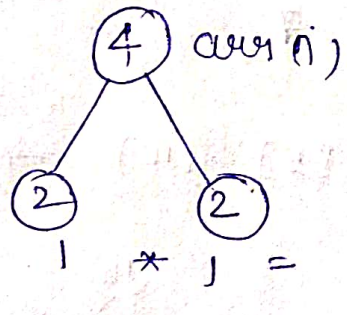
4 X 3 ⇒ 12

(14) Longest Increasing Subsequence.

{2, 4, 8}
 j i

map

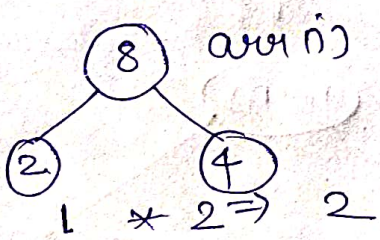
2	1
4	1+1 => 2
8	1



{2, 4, 8}
 j i

map

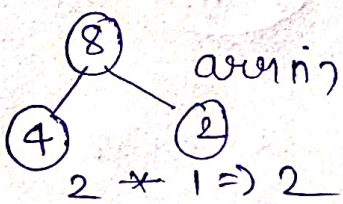
2	1
4	2
8	1+2 => 3



{2, 4, 8}
 j i

map

2	1
4	2
8	3+2 => 5



{2, 4, 8}
 j i ✗ j < i

map

2	1
4	2
8	5

(1) Sort
(2) map
(3) i = 1, i < n
(4) j = 0, j < i } arr[i] = root
left = arr[j]
root/left == 0
Right = root/left
↳ map
(5) Multiply.

// Code for Binary Trees With Factors -

```
public class Solution {
    private static final int MOD = 1000000007;

    public int numFactoredBinaryTrees(int[] arr) {
        int n = arr.length;
        Arrays.sort(arr);

        Map<Integer, Long> mp = new HashMap<>();
        mp.put(arr[0], 1L);

        for (int i = 1; i < n; i++) {
            long count = 1;

            for (int j = 0; j < i; j++) {
                int v = arr[j];

                if (arr[i] % v == 0 && mp.containsKey(arr[i] / v)) {
                    count += (mp.get(v) * mp.get(arr[i] / v))
                             % MOD;

                    count %= MOD;
                }
            }

            mp.put(arr[i], count);
        }

        int result = 0;
        for (long val : mp.values()) {
            result = (int) ((result + val) % MOD);
        }

        return result;
    }
}
```

M5

(08) Restore the Array from Adjacent Pairs –

Examples ⇒ adjacentPairs - {{2,1}, {3,4}, {3,2}}

output – {1, 2, 3, 4}
OR
{4, 3, 2, 1}

– DRY RUN –



Adjacency List है ये Bro...

map	adj
2	{1,3}
1	2
3	{2,4}
4	3

यहाँ पर tension free है 1 और 4 क्योंकि इनके एक ही neighbour so Start with any of them.

if ⇒ 1 → 2 → 3 → 4

if ⇒ 4 → 3 → 2 → 1

① Adj
② Start Point → 1 neighbour

③ DFS

16

```
public class Solution {
    Map<Integer, List<Integer>> adj = new HashMap<>();

    public int[] restoreArray(int[][] adjacentPairs) {
        for (int[] edge : adjacentPairs) {
            int x = edge[0];
            int y = edge[1];

            adj.computeIfAbsent(x, k -> new ArrayList<>()).add(y);
            adj.computeIfAbsent(y, k -> new ArrayList<>()).add(x);
        }

        int root = 0;
        for (int num : adj.keySet()) {
            if (adj.get(num).size() == 1) {
                root = num;
                break;
            }
        }

        int[] result = new int[adj.size()];
        dfs(root, Integer.MAX_VALUE, result, 0);
        return result;
    }

    private void dfs(int node, int prev, int[] result, int i) {
        result[i] = node;
        for (int neighbor : adj.get(node)) {
            if (neighbor != prev) {
                dfs(neighbor, node, result, i+1);
            }
        }
    }
}
```

(17)

(09) Array Pair Sum Divisibility Problem –

Ex. nums = {9, 5, 7, 3}

K = 6

Ans. True

	0	1	2	3
	F	F	F	F

// Brute force –

```
for (i=0; i<n; i++) {
    if (used[i] == True) continue;
    for (j = i+1; j<n; j++) {
        if (used[j] == True) continue;
        if ((nums[i] + nums[j]) % K == 0) {
            used[i] = True;
            used[j] = True;
            foundPair = True;
            break;
        }
    }
    if (foundPair == false) {
        return false;
    }
}
return True;
}
```

T.L.E.

TC – $O(n^2)$

SC – $O(n)$

(18)

//Optimal —

Corner Case :- {6, 12} K = 6

r1 = 12 % K = 12 % 6 = 0



a % K = r1

b % K = r2

(a+b) % K == 0

r1 + r2 == K

Code :-

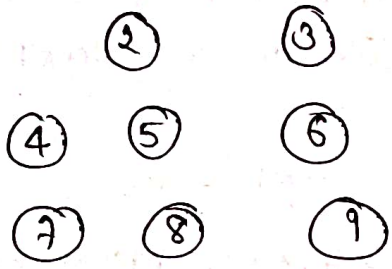
```
public boolean canPair (int nums[], int k){
    int n = nums.length;
    if( n % 2 == 1){
        return false;
    }
    Set<Integer> st = new HashSet<>();
    for (int i=0; i<n; i++){
        int remain = nums[i] % k;
        int other_remain = k - remain;
        if (st.contains (other_remain) ||
                ( (remain == 0) && (st.contains (0)))) {
            st.remove (remain);
            st.remove (other_remain);
        }
        else {
            st.add (remain);
        }
    }
    return st.size () == 0;
}
```

(19)

(10) Minimum Number of Pushes to Type Word - I

Example :- word = "abcde"

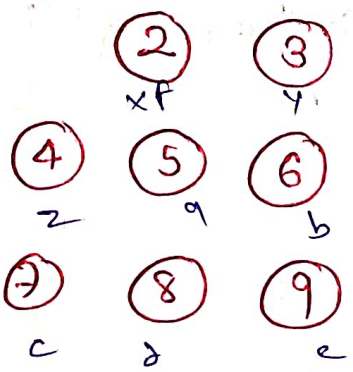
output = 5



Dry Run :-

word = "xyzabcdef"

output 10



assign_key = 2

key

key	
2	1+1
3	1
4	1
5	1
6	1
7	1
8	1

20) Code:-

```
public int minimumPushes (String word) {
    if (word.length() <= 8) {
        return word.length();
    }
    int count = 0;
    Map<Integer, Integer> mp = new HashMap<>();
    int assign = 2;
    for (char ch : word.toCharArray()) {
        if (assign > 9) {
            assign = 2;
        }
        mp.put(assign, mp.getOrDefault(assign, 0) + 1);
        count += mp.get(assign);
        assign++;
    }
    return count;
}
```

```
public int minimumPushes (String word) {
    int mp[] = new int[26];
    for (char ch : word.toCharArray()) {
        mp[ch - 'a'] = 1;
    }
    Integer mpInteger[] = Arrays.stream(mp).boxed().
                          toArray(Integer[]::new);
    Arrays.sort(mpInteger, Comparator.reverseOrder());
    int ans = 0;
    for (int i = 0; i < 26; i++) {
        ans += mpInteger[i] * ((i/8) + 1);
    }
    return ans;
}
```

21) Minimum Number of Pushes To type Word II

Ex. word = "a iiiiiii"

Q

— Dry Run —

(2) (2)

qi iq

1+2+2+2+2+2+2 2+1+1+1+1+1+1 = 8

⇒ 13 > 8

(22) Code -

```
public int minimumPushes (String word) {
    int freq[] = new int[26];
    for (char ch: word.toCharArray()) {
        freq[ch - 'a']++;
    }
    Arrays.sort(freq);

    int pushes = 0;
    int position = 0;

    for (int i = 25; i >= 0; i--) {
        if (freq[i] == 0) {
            break;
        }
        pushes += freq[i] * (position / 8) + 1);
        position++;
    }
    return pushes;
}
```

(11) Hand of Straights -

Ex: hand = {1, 2, 3, 6, 2, 3, 4, 7, 8}

groupSize = 3

output - true

- Dry Run -

curr (1) : {2, 3}

curr = 6 {6, 7, 8}

true.

map

num	freq.
1	✗ 0
2	2 ✗ 0
3	2 ✗ 0
4	✗ 0
6	✗ 0
7	✗ 0
8	✗ 0

(24) Code:-

```
public boolean isNStraightHand(int hand[], int groupSize){
    int n = hand.length;

    if(n % groupSize != 0){
        return false;
    }

    TreeMap<Integer, Integer> map = new TreeMap<>();

    for(int card: hand){
        map.put(card, map.getOrDefault(card, 0) + 1);
    }

    while(!map.isEmpty()){
        int curr = map.firstKey();

        for(int i = 0; i < groupSize; i++){
            if(!map.containsKey(curr+i)){
                return false;
            }

            map.put(curr+i, map.get(curr+i)-1);
            if(map.get(curr+i) == 0){
                map.remove(curr+i);
            }
        }
    }

    return true;
}
```

(12) Replace Words -

Ex. dictionary = {"cat", "bat", "rat"}

sentence = the cattle was rattled by the battery

output - "the cat was rat by the bat"

- Dry Run -

(1) Sentence → each (word) → stringstream

(2) rattled

25) Code -

```
private String findRoot(String word, Set<String> st){
    for(int l=1; l<=word.length(); l++){
        String root = word.substring(0, l);
        if(st.contains(root)){
            return root;
        }
    }
    return word;
}
public String replaceWords(List<String> dictionary,
                           String sentence){
    Set<String> st = new HashSet<>(dictionary);

    StringBuilder result = new StringBuilder();
    String[] words = sentence.split(" ");

    for(String word: words){
        result.append(findRoot(word, st)).append(" ");
    }

    return result.toString().trim();
}
```